

Article

Digital Twin-Enhanced Programming Education: An Empirical Study on Learning Engagement and Skill Acquisition

Ming Lu *  and Zhongyi Hu

College of Computer Science and Artificial Intelligence, Wenzhou University, Wenzhou 325035, China

* Correspondence: 20210069@wzu.edu.cn

Abstract

As an introductory core course in computer science and related fields, “Fundamentals of Programming” has always faced many challenges in stimulating students’ interest in learning and cultivating their practical coding abilities. The traditional teaching model often fails to effectively connect theoretical knowledge with practical applications, resulting in a low retention rate of students’ learning and a weak ability to solve practical problems. Digital twin (DT) technology offers a novel approach to addressing these challenges by creating dynamic, virtual replicas of physical systems with real-time, interactive capabilities. This study explores DT integration in programming teaching and its impact on learning engagement (behavioral, cognitive, emotional) and skill acquisition (syntax, algorithm design, debugging). A quasi-experimental design was employed to study 135 first-year undergraduate students, divided into an experimental group ($n = 90$) using a DT-based learning environment and a control group ($n = 45$) receiving traditional instruction. Quantitative data analysis was conducted on participation surveys, planning evaluations, and qualitative feedback. The results showed that, compared with the control group, the DT group exhibited a higher level of sustained participation ($p < 0.01$) and achieved better results in actual coding tasks ($p < 0.05$). Students with limited coding experience showed the most significant progress in algorithmic thinking. The findings highlight that digital twin technology significantly enhances engagement and skill acquisition in introductory programming, particularly benefiting novice learners through immersive, theory-aligned experiences. This study establishes a new paradigm for introductory programming education by addressing two critical gaps in digital twin applications: (1) differential effects on students with varying prior knowledge (engagement/skill acquisition) and (2) pedagogical mechanisms in conceptual visualization and authentic context creation.

Keywords: digital twin; programming education; learning engagement; skill acquisition; immersive education; educational technology



Academic Editor: Stelios Xinogalos

Received: 19 July 2025

Revised: 5 August 2025

Accepted: 7 August 2025

Published: 8 August 2025

Citation: Lu, M.; Hu, Z. Digital Twin-Enhanced Programming Education: An Empirical Study on Learning Engagement and Skill Acquisition. *Computers* **2025**, *14*, 322. <https://doi.org/10.3390/computers14080322>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The course “Fundamentals of Programming” is the cornerstone of computer science education, aiming to cultivate students’ logical thinking abilities, grammatical application skills, and basic coding skills [1]. It lays a solid foundation for subsequent advanced courses, such as data structures, algorithms, and software engineering. Therefore, the teaching effectiveness of this course has a significant impact on students’ long-term academic development and their success in their career paths [2]. However, education statistics present a worrying situation: globally, over 40% of students struggle to complete this

course, and in many educational institutions, the dropout rate for this course exceeds 30% [3]. There are three main reasons for these results:

Firstly, students find it challenging to visualize the execution process of code (reported by 32% of students). Programming concepts such as loops, conditions, and function calls are essentially abstract. Students often find it challenging to map lines of code to their tangible results in their minds [4]. For example, beginners may write a “For loop” but cannot master how to iterate over the dataset, leading to confusion during debugging. Secondly, there is a lack of practical application scenarios in learning (reported by 28% of students) [3]. Traditional teaching methods are often limited to isolated coding exercises, such as “calculating the factorial of numbers”, which can make students feel disconnected from the real world. This sense of disconnection will weaken students’ understanding of the practical value of the course theme, thereby reducing their motivation to engage deeply in the learning process [5].

Third, there is inadequate real-time feedback (reported by 21% of students) [3]. In traditional settings, feedback on code often comes hours or days after submission, in the form of cryptic compiler messages or brief instructor comments. This delay hinders the iterative learning process, as students are unable to associate errors with specific concepts [6] immediately.

These challenges fully demonstrate the urgent need for innovative teaching methods to bridge the gap between abstract programming concepts and concrete, practical outcomes. Digital twin (DT) technology is a promising solution [7]. A digital twin refers to the virtual representation of physical entities or systems, which enables real-time data exchange and simulation [8]. It has been widely utilized in various industries, including predictive maintenance in manufacturing, surgical simulation in healthcare, and urban planning in innovative city development [9]. In education, its potential lies in creating dynamic, interactive environments where students can manipulate virtual models to observe the consequences of their actions—an attribute particularly valuable for programming education [10].

Unlike static simulations or virtual labs, digital twins offer bidirectional data flow: students’ code inputs immediately affect the virtual system, which in turn provides contextually relevant feedback [11]. For example, a student writing code to control a virtual robot in a DT environment can see the robot stall if a loop lacks a termination condition, with the system flagging, “Infinite loop detected: Robot motor overload”. This immediate, context-rich feedback bridges the gap between code and outcome, making abstract concepts concrete [12].

The existing educational technology research in programming education mainly focuses on two areas: visual programming tools (such as Scratch [13]) and gamified platforms (such as CodeCombat [14]). Although these tools enhance initial engagement, they often lack the complexity and real-world relevance required to develop powerful programming skills. Visual tools simplify syntax but may not adequately prepare students for text-based coding, and gamified platforms often prioritize competition over a deep conceptual understanding. Digital twins, by contrast, can simulate authentic computing environments (e.g., embedded systems, software architectures) within a controlled educational framework, enabling students to experiment with code in scenarios mirroring professional practice [10]. For instance, a DT of a smart home system can teach students how to program sensor inputs to trigger lighting adjustments—an application with clear real-world relevance.

This study addresses three key research gaps:

1. There is a lack of empirical evidence on digital twin integration in introductory programming courses. While DTs have been explored in advanced computer science

courses (e.g., software engineering [15]), their application in foundational programming instruction remains underexamined.

2. There is an unclear impact of DT-based environments on learning engagement across diverse student backgrounds. Prior research on educational technology often focuses on average outcomes, but little is known about how DTs affect students with varying levels of previous coding experience or learning styles.
3. The potential of digital twins in cultivating algorithmic thinking and error debugging abilities has not been fully developed. These skills are essential for programming ability; however, there is a lack of quantitative research on how interactive and feedback-rich environments, such as digital twins, affect the development of these skills.

To fill these research gaps, this study aims to provide a theoretical basis and practical guidance for educators who wish to apply digital twin technology in programming education. Specifically, this study attempts to answer the following questions: How will digital twins affect students' engagement (including behavioral, cognitive, and emotional aspects) in the "Fundamentals of Programming" course? What impact will it have on students' acquisition of core programming skills? Are there differences in these impacts among different student groups?

In order to clarify how digital twin technology enhances programming teaching, the relevant terms are introduced according to the logical chain of "core scenario → system architecture → implementation tools" to facilitate understanding in subsequent chapters.

Virtual smart classrooms: They are interactive learning environments created through digital twin technology. In this study, a virtual space was constructed using 3D rendering, which includes interactive objects such as robots and sensors. The execution results of student code can be visually presented through the state changes of these objects, and programming concepts can be visualized through the use of 3D rendering.

Digital twin teaching system (DT teaching system): It is an architecture composed of three interrelated layers—the client layer, the middleware layer, and the server layer. The client layer provides students with an interface to interact with the digital twin environment, including components like a 3D rendering engine for immersive visualization and a code editor for writing code. The middleware layer acts as a bridge, processing code, generating simulations, and enabling real-time communication. The server layer manages data persistence, user progress, and collaborative features. This system can transform abstract programming concepts into visual representations, provide real-time and scenario-specific feedback, and promote interactive and collaborative learning. It supports the operation of the virtual smart classroom.

Digital twin platform (DT platform): It is a platform that uses Unity3D to build a 3D visual interface and combines Python to implement backend logic. In this study, it has created a reproduction scene of a virtual smart classroom. This platform includes a code interface with syntax highlighting and real-time syntax checking capabilities, a real-time simulation function to visually present the program execution process, a feedback system to provide contextual prompts, and a collaborative space for students to interact and learn from each other.

2. Literature Review

2.1. Digital Twin Technology

Digital twin (DT) technology originated in the early 2000s, with NASA's use of virtual replicas to monitor and simulate the performance of spacecraft [16]. Over the past two decades, advancements in cloud computing, the Internet of Things (IoT), and 3D modeling have expanded their applications [8]. DT technology constructs dynamic virtual replicas of physical

entities and enables real-time data interaction between physical and virtual spaces [17]. The core of digital twins consists of three parts: (1) physical entities (or logical systems), (2) virtual replicas, and (3) data bridges for real-time synchronization [18]. This tripartite structure enables iterative testing, predictive analysis, and scenario simulation—features that make DT invaluable to education [7]. In this study, a digital twin (DT) refers to a dynamic virtual replica of physical programming tools (e.g., robots, sensors) that synchronizes with code inputs to simulate real-world behavior within a virtual smart classroom. The virtual smart classroom is defined as a comprehensive teaching environment that integrates multiple DTs and support systems.

In technical education, traditional programming teaching tools, such as CodePen [19] and JSFiddle [20], focus on code editing and basic visual preview but lack dynamic visualization of the underlying mechanisms. This has led to a lack of mechanisms in teaching to reduce the cognitive load of abstract concepts. DT technology can tightly couple the establishment of code syntax and visual results, solving this challenge. DT technology has been used to teach complex systems. In engineering, students interact with DT replicas of machinery to learn about mechanical stress, with the virtual model visualizing wear and tear as parameters change [21]. In robotics, DT technology enables students to program motion algorithms and test them in a virtual environment before deploying them to physical robots, thereby reducing the risk of equipment damage [22]. In computer science, the study of network systems teaches students about network security knowledge by simulating hacker attacks and defense protocols [23].

DT provides unique advantages for programming education [24]:

- Specificity: Abstract concepts (such as variables and loops) are related to observable results (such as virtual counter increments and the gradual movement of robots).
- Security: Students can experiment with risky code (such as infinite loops) without compromising physical systems or data.
- Context: Embedding encoding tasks into meaningful scenarios (such as programming virtual thermostats) enhances perceptual relevance.

2.2. Learning Participation in Programming Education

Learning engagement encompasses three dimensions, behavior, cognition, and emotion, and is a key factor determining the effectiveness of programming course teaching [25]. Among them, behavioral participation is specifically manifested as actively engaging in various coding tasks, such as writing code, troubleshooting program errors, and collaborating with classmates [26]. Cognitive involvement involves in-depth processing of programming logic (such as analyzing the reasons for function failures and linking new concepts with prior knowledge) [27]. Emotional engagement reflects positive attitudes toward coding challenges (e.g., persistence despite errors, enjoyment of problem-solving) [28].

Traditional teaching methods often struggle to balance cognitive, behavioral, and emotional learning engagement [29]. For example, lecture formats frequently rely on passive cognitive participation, with students only listening to explanations and rarely having the opportunity to apply learned concepts in real time. Conventional laboratory exercises, on the other hand, focus on practical actions at the behavioral level. Students often copy preset code snippets without delving into the underlying logical principles. These two forms frequently overlook emotional involvement, as repeated errors and delayed feedback can lead to frustration and anxiety, a phenomenon known as “programming phobia” [30].

Technological enhancement methods attempt to address these gaps but have limitations. Interactive coding platforms, such as LeetCode, enhance behavioral engagement through real-time syntax validation; however, they rarely promote cognitive engagement in complex logic [31]. Video tutorials have improved passive cognitive engagement but lack

interactivity and limit behavioral practice [32]. Peer code review platforms can enhance students' emotional engagement through social support, but such platforms may not provide the structured feedback necessary for skill development [33]. Digital twin technology possesses the characteristics of immersion and instant response [34], thereby having the potential to mobilize students' enthusiasm for participation at multiple levels.

- Behavioral engagement: Hands-on coding in a virtual environment, where every keystroke affects the DT's state, encourages active participation.
- Cognitive engagement: Real-time visualization of code influence (such as data flowing through loops) facilitates deep processing of logic.
- Emotional participation: The digital twin environment provides a safe experimental space, where even if errors occur, they will only result in virtual system failures rather than real-life failures in public settings. This helps alleviate students' anxiety and cultivates their resilience when facing problems.

2.3. Skill Acquisition in Introductory Programming

The core skills targeted in "Fundamentals of Programming" include mastery of syntax, algorithmic thinking, and proficiency in debugging. Research indicates that these skills develop most effectively through deliberate practice with immediate feedback [35]. This process aligns with the "testing effect" in cognitive psychology, whereby active retrieval and application strengthen memory retention [36].

Traditional teaching feedback mechanisms often hinder students' acquisition of skills [37]. Compiler messages (e.g., "Syntax error on line 5") are technical and decontextualized, failing to explain why the error occurred or how to fix it. Instructor reviews are delayed, disrupting the connection between action (coding) and consequence (feedback). Peer feedback may be inaccurate, as novices often lack the expertise to identify logical errors.

In this study, feedback mechanisms are clarified as a collaboration between the DT (virtual robot's behavioral response) and the DT teaching system (contextual error prompts), which greatly improves learning skills.

Digital twin environments address the above limitations through three key features [10]:

- Real-time syntax validation: As students write code, the DT teaching system (middleware layer) checks for errors (e.g., missing semicolons, undeclared variables) and highlights them in the context of the virtual system's behavior. For example, a missing closing brace in a function may cause a virtual sensor to "malfunction" with the DT annotating, "Function incomplete: Sensor data not processed".
- Algorithm visualization: The DT teaching system (middleware layer) visually illustrates how data flows within the program structure. Students who program sorting algorithms can observe virtual systems rearranging numbers in real time, making it easier to identify inefficiencies (such as unnecessary iterations).
- Root-cause debugging: Students can manipulate the DT's state (e.g., resetting input values, pausing execution) to trace errors back to specific code segments. For instance, a logic error in a conditional statement may cause a virtual door to "open when it should close"; by adjusting the condition and observing the DT's response, students learn to isolate the mistake.

These features are consistent with the cognitive load theory [38], which emphasizes the importance of manageable information presentation in skill acquisition. By allocating information through visual (DT simulation), textual (code), and contextual (feedback) channels, the DT reduces the mental effort required to process complex concepts, thereby releasing cognitive resources for deep learning.

3. Research Design and Methodology

3.1. Research Questions

This study aims to answer the following research questions:

1. Compared with traditional methods, does integrating digital twin technology in “Fundamentals of Programming” teaching significantly improve students’ learning engagement (behavior, cognition, emotion)?
2. What impact does a DT-based learning environment have on students’ acquisition of core programming skills (syntax application, algorithm design, debugging)?
3. Are there differences in the effectiveness of DT technology across student subgroups with varying prior coding experience?

3.2. Participants

The participants are 135 first-year undergraduate students enrolled in the “Fundamentals of Programming” course at a public university in Wenzhou for the 2024–2025 academic year. This sample comes from three complete classes, and the participants mainly learn the following:

- Computer science (62%, $n = 84$);
- Information science (23%, $n = 31$);
- Related fields (such as software engineering, data science) (15%, $n = 20$).

The experiment selected 89 males (65.9%) and 46 females (34.1%), reflecting the typical gender distribution of computer science majors in the region [39]. The participants were aged between 18 and 21 years old ($M = 19.2$, $SD = 0.8$), with 92% of them being between 18 and 19 years old (first-year undergraduate students). All participants reported no formal programming training, confirmed via a pre-course questionnaire. Self-reported exposure to coding (e.g., online tutorials) was minimal, with a mean of 1.2 h/week ($SD = 0.7$), and no significant differences were found between groups ($p > 0.05$).

To ensure internal effectiveness, the participants were randomly assigned to either the experimental group ($n = 90$) or the control group ($n = 45$) after completing the pre-class aptitude test. This test evaluates basic programming skills, including logical reasoning (such as pattern recognition and syllogism), as well as fundamental mathematical skills (such as algebra and problem-solving). The independent sample t-test confirmed that there was no significant difference in the scores of the aptitude test between the two groups, ensuring consistency in the initial ability level.

The basic information of the participants is shown in Table 1. It presents a comparison of baseline data on key features between the experimental group (digital twin teaching group) and the control group (traditional teaching group), with statistical significance mainly reflected in the following aspects.

(1) Verifying group balance

The intergroup differences in all features satisfy $p > 0.05$ (with the lowest being 0.17 for extracurricular coding time), indicating that there is no statistically significant difference between the two groups in core variables such as gender ratio, age distribution, professional composition, basic logical reasoning ability, and extracurricular coding experience. This proves that random grouping is effective, and the initial states of the two groups of students are comparable, eliminating the interference of baseline differences for the attribution of subsequent intervention effects (such as learning engagement and skill improvement).

(2) Controlling confounding variables

The balance of demographic characteristics, such as gender, age, and significance, ensures that these factors do not systematically affect the experimental results (such as potential differences in programming learning due to different professional backgrounds).

There was no significant difference ($p = 0.68$) in the pre-test logical reasoning score (a key foundational ability for programming learning), indicating that the two groups of students have the same starting point for core learning abilities, and subsequent skill improvement is more likely to be attributed to the digital twin teaching model rather than initial ability differences.

The balance of extracurricular coding time ($p = 0.17$) eliminates the confusion caused by “extra practice time differences” in the experimental results.

(3) Enhancing research validity

The balance of baseline data is a crucial guarantee of internal validity in quasi-experimental designs. These data support the conclusion that the differences between the experimental group and the control group after intervention, such as skill scores and engagement levels, are more likely to be caused by the digital twin teaching model rather than inherent differences between the two groups of students, thereby strengthening the credibility of the research findings.

This study belongs to the scope of university teaching reform research and is an integral part of the conventional teaching process. Participants in the study were informed of the study details and their informed consent was obtained. They had the right to withdraw from the study at any time without incurring any penalties or adverse consequences. The research data were anonymized using a unique identifier, and all records were securely stored in an encrypted database.

Table 1. Participant basic information form.

Characteristic Indicators	Experimental Group ($n = 90$)	Control Group ($n = 45$)	Intergroup Differences (p -Value)
Gender (Male/Female)	57/33	32/13	0.89
Age ($M \pm SD$, years)	19.1 ± 0.7	19.3 ± 0.9	0.23
Professional distribution (%)			
Computer science (CS)	CS: 61.1	CS: 62.2	0.91
Information science (IS)	IS: 22.2	IS: 24.4	
Related fields (RFs)	RFs: 16.7	RFs: 13.4	
Pre-test logical reasoning score ($M \pm SD$)	68.3 ± 7.2	67.8 ± 7.5	0.68
Weekly extracurricular coding time ($M \pm SD$, hours)	1.3 ± 0.6	1.1 ± 0.8	0.17

3.3. Intervention Design

Two groups of students participated in an eight-week course centered on the core theme of “Fundamentals of Programming”. The specific arrangement is as follows: In the first two weeks of the course, students mainly learned about the definition and use of variables, the characteristics and applicable scenarios of different data types, and basic input and output operations. Through these contents, they established a preliminary understanding of data processing in programming. Entering weeks 3 to 4, the course focuses on the study of control structures, including the logical judgment and application of if-else conditional statements and the use of loop statements such as for and while, to help students master the skills of implementing different process controls through code. In weeks 5 to 6, the teaching focus shifts to the definition, calling, and parameter passing rules of functions, allowing students to understand how to encapsulate code blocks through functions and improve code reusability and readability. In the last two weeks of the course, namely weeks 7 and 8, students begin to learn the basic concepts, creation, and operation

methods of arrays, as well as basic debugging skills. During the process of identifying and correcting code errors, students further consolidated their prior knowledge and enhanced their programming practical skills. The total teaching time for the two groups of students is the same, at 24 h, with lectures and laboratory practice each accounting for 12 h. The core difference between the two is reflected in the laboratory activities—the control group adopts traditional teaching methods, while the experimental group incorporates digital twin technology.

3.3.1. Architecture of DT Teaching System

As shown in Figure 1, the DT teaching system’s architecture comprises three inter-related layers: the client layer, the middleware layer, and the server layer. Each layer is equipped with specialized components designed to achieve real-time interaction, simulation operations, and collaborative learning in the programming teaching process.

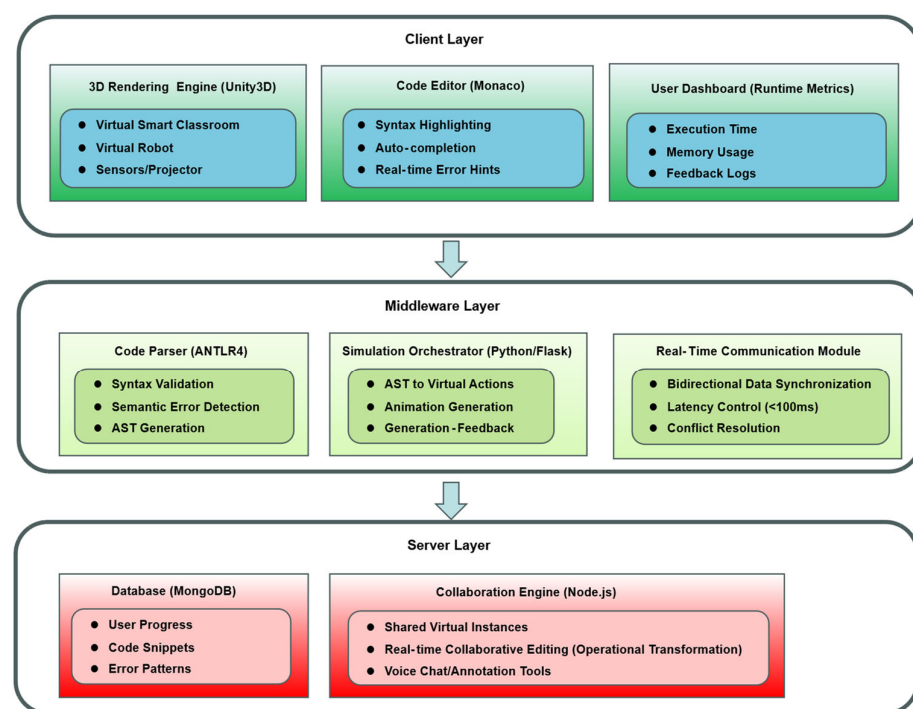


Figure 1. Architecture of the DT teaching system.

(1) Client Layer

The client layer serves as the user interface, providing students with a direct channel to interact with the digital twin environment. The original intention of this layer’s design is to create an immersive and intuitive user experience, closely linking the code-writing process with visual feedback.

- **3D Rendering Engine (Unity3D)**

It renders the “Virtual Smart Classroom”—a 3D environment where programming concepts are visualized through interactive objects. It is a simulated space containing virtual devices (e.g., temperature sensors, projectors, and a robotic assistant) that respond to student code. There is a 3D avatar whose movements (e.g., navigation, object manipulation) are controlled by student-written functions (e.g., `move_robot()`, `turn_left()`), making function calls and parameter passing tangible. Virtual sensors display real-time data (e.g., `current_temp = 25 °C`) to illustrate variables and data flow; the projector animates loop iterations (e.g., counting from 1 to 10) to visualize control structures.

- **Code Editor (Monaco Editor)**
It provides a text-based interface for writing, editing, and submitting code (in the C language, aligned with course textbooks) comprising color-coding keywords, variables, and functions to improve readability. It suggests using commonly used functions (such as `printf()` and `scanf()`) and variable names to minimize syntax errors. It uses contextual tooltips to emphasize syntax errors (such as missing semicolons) and link them to potential fixes.
- **User Dashboard**
It displays runtime metrics and feedback to help students monitor the performance and progress of their code. It measures the time required for code execution to improve efficiency. It visualizes resource consumption (such as a “virtual memory module with a capacity of 60%”) for optimized education. It records system messages (such as “Infinite loop detected: robot battery low”) to assist with debugging.

(2) Middleware Layer

The middleware layer acts as a bridge between the client and server, processing code, generating simulations, and enabling real-time communication. It ensures that students’ input is converted into virtual actions and feedback. It is critical for translating code to virtual actions, addressing the gap between text and visualization.

- **Code Parser (ANTLR4)**
It validates code syntax and semantics for simulation purposes by converting text-based code into a structured format. Grammar validation involves checking for structural errors (such as missing curly braces or incorrect loop syntax) and marking them to the client. Logical issues (such as undeclared variables or mismatched parameters) are identified and correlated with virtual system behavior (e.g., undeclared “speed” variable results in no response from robot movement). The valid code is then converted into an Abstract Syntax Tree (AST), a hierarchical representation of the code logic, for processing by the simulation orchestrator.
- **Simulation Orchestrator (Python/Flask)**
It maps an AST onto virtual actions, generating animations and feedback according to code logic. It transforms code structures into 3D animations (e.g., for loop nodes triggering rotation counters and `printf` statements updating a virtual projector). It creates real-time visual effects (such as robot movements, sensor data updates) to visualize code execution. Additionally, it generates contextually rich messages about virtual system behavior (e.g., the “`calculate_average()`” function lacks a return value, and the sensor displays “undefined”).
- **Real-Time Communication Module**
It supports bidirectional data transmission between the client and server, ensuring synchronization of code editing, simulation, and feedback. For bidirectional data synchronization, it transfers code submissions from the client to the server and sends simulation results/feedback back to the client. It maintains a latency of less than 100 ms to ensure a sense of “real-time” interaction between code editing and a virtual response. It manages synchronous editing in collaborative mode (e.g., preventing overlapping ping changes to the same line of code).

(3) Server Layer

The server layer manages data persistence, user progress, and collaborative features, supporting long-term learning tracking and peer interaction.

- **Database (MongoDB)**
It stores student data, code snippets, and system logs for progress tracking and adaptive learning. It records task completion status, test scores, and standard error patterns (e.g., students frequently miss loop termination conditions). It saves the submitted code for the instructor's review and future reference. It summarizes common errors to inform teaching (e.g., 30% of students encounter difficulties in array indexing) and tailors feedback accordingly.
- **Collaboration Engine (Node.js)**
This feature fosters multi-user interaction in the digital twin environment, creating an environment that promotes peer learning and team collaboration. Students can invite their classmates to view and edit in the virtual smart classroom, thereby facilitating collaborative debugging and learning. At the same time, with the help of the Operational Translation (OT) algorithm, the code-editing operations of different users are synchronized to ensure that everyone can see the latest code version in real time. This support includes verbal discussions and visual annotations (such as drawing arrows to highlight code errors) to enhance peer feedback. The system operates continuously in a loop, and students submit their work through the client's code editor. The code parser validates syntax/semantics and generates an AST. The simulation orchestrator converts the AST into virtual actions (animations) and feedback. The client renders the animations and displays feedback to the student. The relevant data (including code, learning progress, error records, etc.) are stored in the server database, and collaborative editing content is also synchronized among users. This architecture design can transform abstract programming concepts into visual representations, provide real-time and scenario-specific feedback, promote interactive and collaborative learning, and solve key challenges in entry-level programming education.

3.3.2. Control Group: Traditional Instruction

The control group followed a conventional teaching model, representative of mainstream programming education.

Lectures: There were instructor-centered sessions using static PowerPoint slides and live code demonstrations via a classroom projector. For example, when teaching loops, the instructor displayed a slide with syntax rules (e.g., "for (initialization; condition; increment) {...}") and demonstrated a simple loop to print numbers 1–10. Students copied code snippets into notebooks for later reference.

Lab activities: There was individual practice using a standard online coding platform. Tasks included replicating predefined code (e.g., "Write a for loop to calculate the sum of 1–50") and solving textbook-style problems (e.g., "Find the maximum value in an array"). Feedback was limited to automated syntax checks and a weekly 10 min meeting with the instructor to review errors.

Materials: A printed course packet with code examples, a textbook, and PDF tutorials with step-by-step instructions for each lab task were provided.

The standard online coding platform used by the control group is fundamentally distinct from the DT teaching system, with differences extending far beyond the absence of a 3D rendering engine and real-time communication tools. Its key characteristics and contrasts with the DT system are as follows:

Core functionality: The control group's platform is a text-based coding environment focused solely on syntax validation and code execution (e.g., compiling and running code snippets). It lacks the DT teaching system's ability to map code logic to tangible virtual behaviors (e.g., robot movements, sensor data changes).

Feedback mechanism: Unlike the DT teaching system’s contextual, real-time feedback (e.g., “Infinite loop detected: Robot battery low” linked to virtual robot stalling), the control platform provides only basic, decontextualized feedback (e.g., “Syntax error on line 5”) without explaining the logical or practical implications.

Learning scenarios: The control platform uses isolated, abstract tasks (e.g., “Calculate the sum of 1–50”), whereas the DT teaching system embeds coding in meaningful scenarios (e.g., programming a virtual robot to navigate obstacles in a virtual smart classroom), enhancing relevance to real-world applications.

Interaction and collaboration: The control platform supports only individual coding practice with no collaborative features. In contrast, the DT system includes a shared virtual space for synchronous code editing, voice chat, and peer annotation, fostering collaborative problem-solving.

Concept visualization: Critical to programming education, the DT teaching system visualizes abstract concepts (e.g., loop iterations via animated counters), while the control platform relies on text-only output, leaving logic visualization to the student’s imagination.

In summary, the control group’s platform is a basic tool for code execution, whereas the DT teaching system is an integrated environment that combines virtual replicas (DTs), contextual feedback, scenario-based learning, and collaboration, addressing the limitations of traditional tools in connecting abstract code to practical understanding.

3.3.3. Experimental Group: Digital Twin-Based Instruction

The experimental group received the same lectures and covered the duplicate content as the control group but replaced 50% of lab time (6 h) with DT-based activities. The remaining 50% of lab time involved traditional coding practice to ensure parity in total hands-on experience. The specific differences between digital twin and traditional teaching activities are presented in Table 2, which provides an intuitive comparison of the core differences between the two teaching modes and highlights the innovative aspects of digital twin teaching.

Table 2. Comparison table of digital twins and traditional teaching activities.

Teaching Steps	Experimental Group (Digital Twin Teaching)	Control Group (Traditional Teaching)
Theoretical learning	Same lecturer teaching, consistent content (variables, loops, functions, etc.)	Same lecturer teaching, consistent content
Practical activities (proportion)	50% digital twin tasks 50% traditional exercises	100% Traditional Practice
Core tools	Virtual smart classroom (including code editor, 3D simulator, feedback system)	Text editor Online question answering system
Feedback mechanism	Real-time contextualized feedback (such as “Robot jamming: loop missing termination condition”)	Delayed feedback (automatic grammar check, weekly instructor review)
Collaborative support	Virtual shared space (synchronous editing, voice discussion)	Unstructured collaboration, independently completing tasks
Typical task examples	Programming Control of Virtual Robot Obstacle Avoidance (Integrated Application Loop and conditional statements)	Writing a function to calculate the average value of an array (isolated exercise)

3.3.4. Digital Twin Learning Environment

The DT platform uses Unity3D (Version: Unity 2022.3.62f1 LTS, Unity Technologies, San Francisco, CA, USA) to build a 3D visual interface, combined with Python (Version: 3.13.5, Python Software Foundation, Beaverton, OR, USA) to implement backend logic, to create a reproduction scene of the virtual smart classroom—this is an interactive system

controlled by C language code (consistent with the course material content), including virtual robots, feedback prompt boxes, collaborative editing areas, and other interactive elements. The specific interaction scene is illustrated in Figure 2. The core components of the DT platform are as follows:

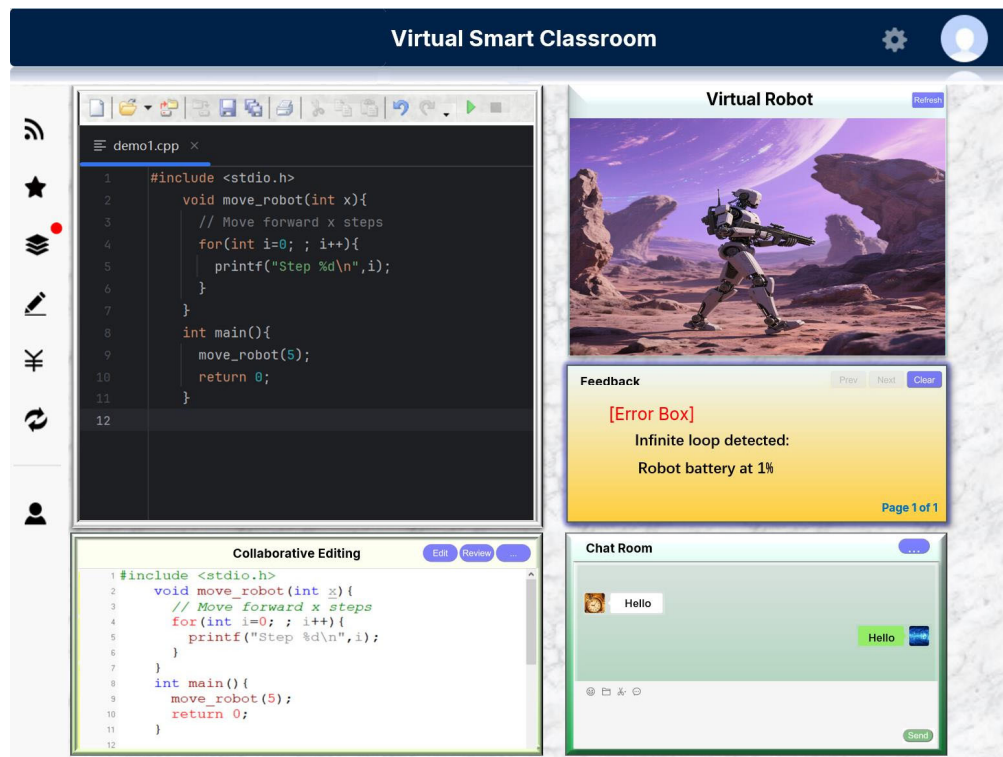


Figure 2. Interactive scene diagram of virtual smart classroom.

Firstly, the code interface, which is a syntax highlighting editor embedded in the digital twin environment, supports writing in the C language. It not only provides an auto-completion function for standard functions such as `printf()` and `scanf()` but also has real-time syntax checking capability. Once an error occurs, it will be highlighted in red.

Next is the real-time simulation function. The virtual smart classroom can visually present the program execution process; for example, loop iterations will be displayed in the form of animated loops. For example, the virtual counter corresponding to the “for loop” will increase from 1 to 10, and the changes in variables will be reflected in real time in the virtual memory module. For example, when processing sensor data, the “temperature” variable on the digital display will be updated accordingly, and function calls will trigger visible operations. For example, when executing the “`move_robot()`” function, the robot assistant will move to the specified position.

Furthermore, a feedback system is in place that can provide contextual prompts based on the digital twin’s status. If there is a syntax error, it will prompt “Missing semicolon in line 8: Virtual projector freeze check code structure”. When encountering a logical error, it will display “Detected infinite loop: Robot helper battery level is 1%—Add termination condition”. For areas that can be optimized, suggestions similar to “Redundant variable ‘x’ in line 12: Virtual sensor processing delay, simplify code” will also be given. Of course, such feedback is generated by the DT teaching system (especially its middleware layer, including code parsers and simulation choreographers), while DT technology only exhibits behavioral responses (such as stagnation).

Finally, there is the collaborative space, a shared virtual “laboratory area” where students can invite their classmates to view their digital twin instances. It includes features

such as synchronous code editing, voice chat, and annotation tools, including highlighting errors in peer code by drawing arrows.

3.3.5. Technology Justification

The technical components of the DT teaching system and control platform were selected based on educational suitability, classroom feasibility, and the capacity to address traditional tool limitations.

(1) Unity3D for 3D Visualization

Unity3D was chosen as the core engine for rendering the virtual smart classroom, balancing fidelity and accessibility. Its robust 3D pipeline supports interactive elements critical to programming education, including articulated virtual robots, responsive sensors, and context-aware lighting. Compared to Unreal Engine, Unity3D operates efficiently on standard university hardware (eliminating GPU requirements) and offers pre-built educational assets (e.g., robot controllers) that accelerate development by ~40% without compromising pedagogical value. These features enable visualization of abstract concepts (e.g., loop iterations as robot patrol patterns) in classroom-friendly environments.

(2) Python/Flask for Backend Logic

Python with Flask was selected for backend implementation due to its educational compatibility and prototyping efficiency. The middleware layer leverages Python's ecosystem for real-time code parsing (via ANTLR4 integration) and data visualization (e.g., battery consumption graphs). Unlike Java, Python's concise syntax and dynamic typing facilitated rapid iteration of feedback logic, while Flask's modular design ensured sub-500 ms response times between code submission and virtual environment updates.

(3) LeetCode-like Platform for Control Group

The control group used a text-based coding platform to isolate DT-specific advantages. This tool epitomizes traditional programming education, supporting code execution and syntax checking but lacking visualization. The key limitations include abstract error messages (e.g., "Syntax error" without real-world context), static text output, and decontextualized tasks. These gaps highlight the DT teaching system's unique value in bridging code and tangible outcomes.

3.4. Data Collection Tools

The research data were collected at three time points: before the intervention began (week 1), during the intervention (week 4), and after the intervention ended (week 8). A mixed-method research approach was employed to combine quantitative and qualitative measurement methods, thereby verifying the survey results from multiple perspectives.

3.4.1. Quantitative Measures

(1) Participation questionnaire

Adapted from the Student Engagement Tool (SEI) [40], it consists of 18 items measured from three dimensions. Among them, there are six items for behavioral engagement, such as "I actively participated in coding tasks during my laboratory time"; six items for cognitive engagement, such as "I deeply thought about why my code was effective or failed"; and six items for emotional engagement, such as "I feel confident when I try to solve new coding problems". All items are scored on a 5-point Likert scale (1 = strongly disagree, 5 = strongly agree) [41], which demonstrates high internal consistency in this study. The Cronbach alpha coefficient of the total score is 0.89, and the alpha coefficients of each subscale are 0.89. A score that is between 0.82 and 0.87 is consistent with previous validation research results [40].

(2) Programming evaluation

This is a 90 min practical test used to evaluate three core skills, including syntax application (30 points), which involves writing code with a correct structure, such as proper loop syntax and function declarations. The algorithm design (40 points) consists of creating a logical sequence to solve a problem, such as designing a function to calculate the average value of an array. Debugging (30 points) refers to identifying and fixing errors in pre-written code, such as correcting conditional statements that cannot handle edge situations. Two senior programming instructors jointly developed this test, and the feasibility of its content was reviewed and confirmed by a small group of three computer science education experts. To establish reliability among raters, two teachers independently rated 20% of the tests, with an intragroup correlation coefficient of 0.92. The difference in ratings was resolved through discussion.

(3) Previous coding experience investigation

This questionnaire consists of 5 items used to evaluate students' self-reported coding exposure, such as "How many hours do you spend coding outside of class per week?", and their confidence in basic computer skills, as indicated by statements like "I feel comfortable using a code editor". This survey helps to divide students into different subgroups for analysis.

3.4.2. Qualitative Measures

Although quantitative measurement was completed earlier and took a dominant position, the use of qualitative research still has practical significance [42]. After the intervention, a study was conducted involving 12 participants, with 6 in each group. The interactive session lasted for 30–45 min, exploring topics such as perspectives on the learning environment (e.g., "How does DT [or the traditional laboratory] influence your understanding of loops?"), motivation and incentives (e.g., "When using DT, did you find yourself more interested in coding tasks? Why or why not?"), and challenges and successes (e.g., "What is the most difficult part of learning programming, and how has DT [or traditional methods] assisted you?"). The interviews were recorded, transcribed verbatim, and anonymized.

All participants filled out a digital log every week, recording the time they invested in coding tasks (both in and out of class), concepts they found easy or difficult, and the strategies used to solve problems (such as "visualizing the loop process with digital twins" or "reviewing examples from textbooks"). These logs were submitted through secure online forms, with a response rate of 91% (sample size of 123).

3.5. Data Analysis

Utilize SPSS 26.0 (Version: IBM SPSS Statistics 26.0, IBM Corporation, Armonk, NY, USA) [43] to conduct an analysis of quantitative data and undertake the subsequent statistical tests. Compare the participation scores and evaluation outcomes between the experimental and control groups, after the intervention. Check the changes in the participation level and skills of the experimental group before and after the intervention. Investigate the differences in results between the different subgroups and stratify them based on previous coding experience, as evaluated through a pre-survey. Calculate the effect size (Cohen's d) using thresholds for mild ($d = 0.2$), moderate ($d = 0.5$), and large ($d = 0.8$) effects to quantify the actual significance of group differences. Follow the steps below to use thematic analysis to analyze qualitative data [44].

- (1) Familiarization: Researchers read transcripts and logs repeatedly to identify initial patterns.
- (2) Coding: Labels were assigned to meaningful segments (e.g., "DT visualization clarifies loops", "frustration with compiler messages").

- (3) Theme development: Codes were grouped into overarching themes (e.g., “feedback quality”, “conceptual clarity”).
- (4) Validation: Themes were reviewed against the original data to ensure accuracy, with a second researcher independently coding 20% of the transcripts (inter-rater reliability, $\kappa = 0.83$).

3.6. Critical Analysis of Methodology

The study’s methodological choices involve deliberate trade-offs to balance rigor and real-world applicability.

(1) The study employed a quasi-experimental design due to practical constraints of maintaining class continuity, which meant not all variables could be strictly controlled. However, this design enhanced ecological validity by reflecting real-classroom settings, with baseline equivalence between groups verified through statistical tests to mitigate selection bias.

(2) Potential novelty effects in DT engagement were addressed by collecting mid-intervention data (week 4), which showed sustained higher engagement in the experimental group, indicating effects were not merely due to initial technology novelty.

(3) The sample size ($n = 135$) was justified through power analysis using Cohen’s d . The effect sizes for key outcomes (e.g., overall engagement $d = 1.40$, algorithm design $d = 1.06$) confirmed sufficient statistical power to detect meaningful differences between groups.

4. Results

The experimental data results in this article have high reliability and are supported by various methods and analytical safeguards. This study employed a quasi-experimental design, randomly dividing 135 first-year graduate students into an experimental group ($n = 90$) and a control group ($n = 45$). The pre-class aptitude test revealed no significant difference in the baseline logical reasoning ability between the two groups ($p = 0.68$), ensuring comparability of the groups. This equivalence can minimize the impact of confounding variables, thereby enhancing the credibility of the differences caused by digital twin (DT) interventions after the intervention. The experimental results will be evaluated across four dimensions: student participation, skill mastery, the differences between subgroups, and the qualitative survey results.

4.1. Learning Engagement

The comparison of learning engagement between the experimental group and the control group is shown in Table 3.

Table 3. Comparison of learning engagement between experimental and control groups.

Engagement Dimension	Experimental Group (M $\pm$ SD)	Control Group (M $\pm$ SD)	<i>t</i> -Value	<i>p</i> -Value	Effect Size (Cohen’s <i>d</i>)
Overall	4.12 $\pm$ 0.53	3.27 $\pm$ 0.61	7.83	<0.001	1.40
Mid-intervention	3.89 $\pm$ 0.57	3.21 $\pm$ 0.63	5.92	<0.001	1.06
Behavioral	4.23 $\pm$ 0.48	3.15 $\pm$ 0.59	10.14	<0.001	1.81
Cognitive	4.01 $\pm$ 0.55	3.32 $\pm$ 0.64	6.37	<0.001	1.14
Emotional	4.16 $\pm$ 0.51	3.36 $\pm$ 0.57	7.29	<0.001	1.31
On-task time (minutes/hour)	42.3 $\pm$ 5.8	28.7 $\pm$ 6.2	10.56	<0.001	-

Note: M = mean; SD = standard deviation. Mid-intervention engagement data were collected at week 4. On-task time reflects minutes spent actively coding per lab hour.

The experimental group scored significantly higher in overall engagement at post-intervention ($M = 4.12$, $SD = 0.53$) compared to the control group ($M = 3.27$, $SD = 0.61$) ($t(135) = 7.83$, $p < 0.001$, $d = 1.40$), indicating a significant practical effect. This difference emerged by mid-intervention (week 4), with the experimental group already showing higher engagement ($M = 3.89$, $SD = 0.57$) than the control group ($M = 3.21$, $SD = 0.63$) ($t(135) = 5.92$, $p < 0.001$, $d = 1.06$), suggesting sustained benefits of the DT environment.

Analyzing student participation in the “Fundamentals of Programming” course from various dimensions, Table 1 reveals significant differences between the experimental group (using digital twin technology) and the control group (traditional teaching). The more obvious observation results are shown in Figure 3. It visualizes the differences between the experimental group and the control group, visually presenting the improvement effect of digital twins on participation in different dimensions and enhancing the persuasiveness of the results.

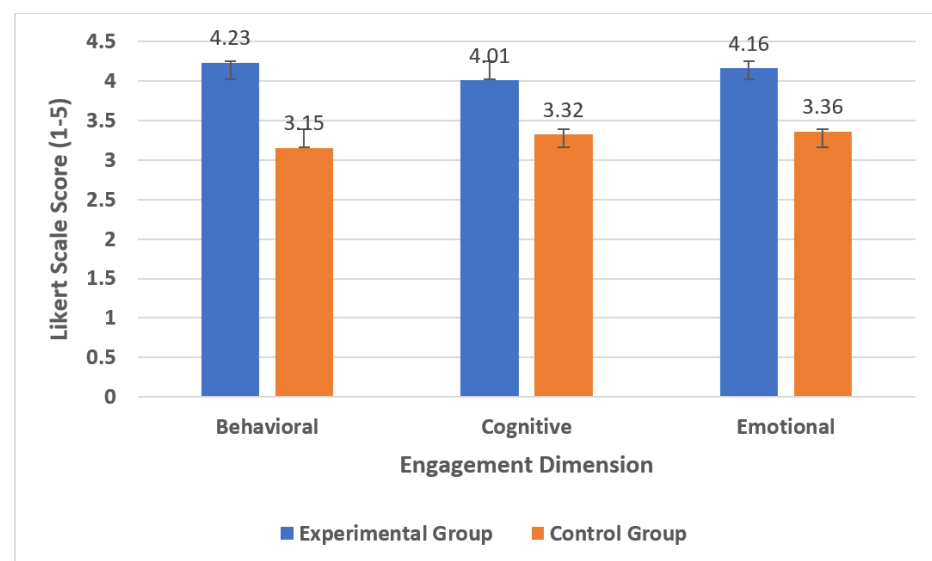


Figure 3. Bar-chart comparing various dimensions of learning engagement.

(1) Behavioral Engagement

The experimental group demonstrated significantly higher active participation, with a mean score of 4.23 ($SD = 0.48$). In learning logs, it was reflected that they spent more time on tasks during labs, averaging 42.3 min per hour ($SD = 5.8$). The interactive nature of the digital twin environment motivated their involvement, as they looked forward to labs because they could see their code directly control the movement of virtual robots, which was more engaging than simply typing code into a blank screen.

Control group: The mean score for behavioral engagement was 3.15 ($SD = 0.59$), and students spent less time on task during labs, averaging 28.7 min per hour ($SD = 6.2$). Their participation was more passive, mainly focused on replicating predefined code or completing isolated exercises.

(2) Cognitive Engagement

Experimental group: Students demonstrated a deeper understanding of programming logic, with a mean score of 4.01 ($SD = 0.55$). Learning logs indicated that they were more likely to reflect on errors, such as analyzing why a loop failed by observing the virtual robot getting stuck. They also asked more conceptual questions (e.g., about the importance of parameter order in functions) rather than just procedural ones.

Control group: The mean score for cognitive engagement was 3.32 (SD = 0.64). Students often expressed confusion about why their code did not work and tended to focus on superficial syntax issues rather than exploring underlying logical principles.

(3) Emotional Engagement

Experimental group: This dimension demonstrated the most significant relative improvement (+23.8% from pre-intervention), with a mean score of 4.16 (SD = 0.51). Eighty-two percent of interviewees mentioned that the “low-stakes experimentation” in the digital twin environment reduced coding anxiety. For example, if they made a mistake, the virtual robot would stop, which did not feel like a significant failure, encouraging them to try harder.

Control group: The mean score for emotional engagement was 3.36 (SD = 0.57). Students reported frustration more frequently, such as disliking seeing “error” messages on the screen without clear explanations of what went wrong.

Overall, the digital twin teaching environment has shown significant effects in enhancing students’ behavior, cognition, and emotional engagement, particularly in terms of emotional participation. This fully demonstrates its potential to create more positive and practical learning experiences.

4.2. Programming Skill Acquisition

The experimental group achieved significantly higher scores on the post-intervention programming assessment ($M = 76.3$, $SD = 10.2$) than the control group ($M = 65.7$, $SD = 11.5$) ($t(135) = 4.59$, $p < 0.001$, $d = 0.82$), indicating a medium-to-large effect size. This advantage was consistent across all three skill dimensions, with the most significant gap in algorithm design.

The comparison of scores in various dimensions of programming skill acquisition is presented in Table 4, which quantitatively illustrates the differences between the two groups in core skills, supporting the conclusion that “digital twins enhance skill acquisition”.

Table 4. Comparison table of scores for various dimensions of programming skills.

Skill Dimension	Experimental Group (M ± SD)	Control Group (M ± SD)	Mean Difference	t-Value	p-Value	Effect Size (d)
Grammar Application (30 points)	25.2 ± 3.8	22.1 ± 4.3	+3.1	3.87	<0.001	0.69
Algorithm Design (40 points)	31.5 ± 5.2	24.8 ± 6.1	+6.7	5.86	<0.001	1.06
Debugging Ability (30 points)	23.7 ± 4.1	18.8 ± 4.7	+4.9	5.02	<0.001	0.91
Comprehensive Score (100 points)	76.3 ± 10.2	65.7 ± 11.5	+10.6	4.59	<0.001	0.82

(1) Syntax application

The experimental group scored higher ($M = 25.2$, $SD = 3.8$) than the control group ($M = 22.1$, $SD = 4.3$) ($t(135) = 3.87$, $p < 0.001$, $d = 0.69$). Instructors noted that the DT’s real-time syntax validation helped students internalize the rules: “Students in the DT group rarely missed semicolons by week 6, because they had learned to associate the red underline with the robot freezing.”

(2) Algorithm design

The most substantial gap occurred in this dimension (experimental $M = 31.5$, $SD = 5.2$; control $M = 24.8$, $SD = 6.1$) ($t(135) = 5.86$, $p < 0.001$, $d = 1.06$). The DT’s visualization of data

flow appeared critical: “Watching the robot follow my algorithm step by step helped me see where the logic broke down” (Student E8). For example, 78% of the experimental group correctly designed a function to sort sensor data, compared to 49% of the control group.

(3) Debugging ability

The experimental group outperformed the control group ($M = 23.7$, $SD = 4.1$ vs. $M = 18.8$, $SD = 4.7$) ($t(135) = 5.02$, $p < 0.001$, $d = 0.91$). DT users were better at tracing errors to their root cause, as noted in learning logs: “Used the DT to pause the code—saw the variable was not updating because I forgot to increment it” (Student E15). Control group students more often relied on trial and error: “I changed random things until the code worked” (Student C12).

The radar chart comparing programming skill scores is shown in Figure 4, which compares the performance of different skill dimensions and highlights the improvement of core skills, such as algorithm design, facilitated by digital twins.

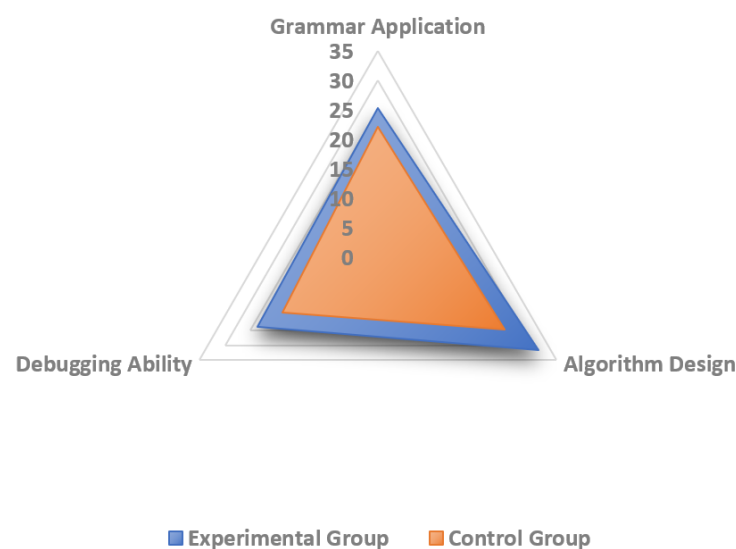


Figure 4. Radar chart comparing programming skill scores.

4.3. Subgroup Differences

Based on their pre-class aptitude test scores, students were divided into three subgroups: low (minimum 30%, $n = 41$), medium (moderate, 40%, $n = 54$), and high (maximum 30%, $n = 40$). Analysis of variance revealed that group–subgroup interactions were significant ($p < 0.05$) in terms of participation and skill level, with the most pronounced benefits observed among students with lower abilities. The comparison of skill improvement among students with different ability levels is shown in Table 5, highlighting the significant improvement effect of digital twins on low-ability students and reflecting the value of educational equity.

Table 5. Comparison table of skill enhancement among students with different ability levels.

Ability Grouping	Experimental Group (Pre-Test → Post-Test)	Increase Amplitude (EG)	Control Group (Pre-Test → Post-Test)	Increase Amplitude (CG)	Effect Size (d)
Low ability ($n = 41$)	52.7 → 64.0	+21.3%	51.9 → 56.4	+8.7%	0.02
Intermediate ability ($n = 54$)	67.3 → 77.8	+15.6%	66.8 → 73.0	+9.2%	0.05
High ability ($n = 40$)	78.5 → 87.0	+10.8%	79.1 → 85.0	+7.5%	0.15

The line charts illustrating skill improvement for students with different abilities in the experimental group are presented in Figure 5. In contrast, those for students with varying

skills in the control group are shown in Figure 6. These displays of changes before and after intervention demonstrate the improvement effect of digital twins on vulnerable groups, supporting the conclusion of “promoting educational equity”.

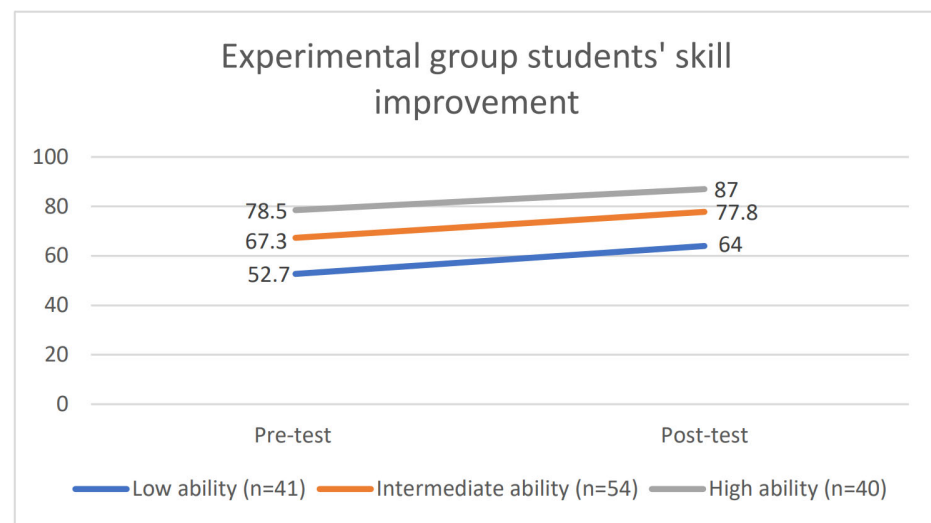


Figure 5. Experimental group students’ skill improvement.

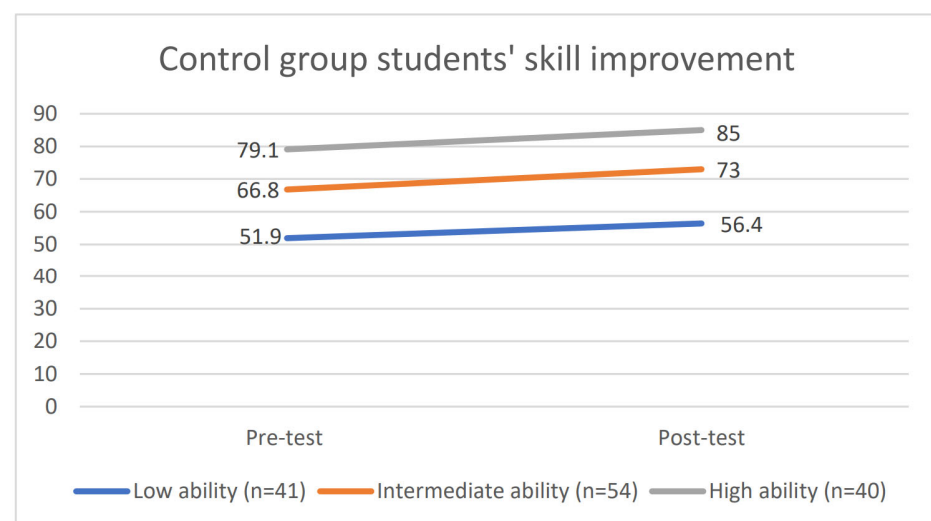


Figure 6. Control group students’ skill improvement.

For the group of low-qualified students, the evaluation scores of the experimental group increased by 21.3% (from a pre-intervention average score of 52.7 to a post-intervention average score of 64.0). In comparison, the evaluation scores of the control group increased by 8.7% (from a pre-intervention average score of 51.9 to a post-intervention average score of 56.4). After testing ($F(1,39) = 5.24, p = 0.02, \eta^2 = 0.12$), a significant difference was found between the two groups. The role of digital twins in helping to understand abstract concepts is evident in the interview, as stated by low-talent student E22: “Before seeing robots counting with my own eyes, I had only a vague understanding of loops—now I finally understand its meaning.”

In the performance of students with moderate abilities, the experimental group improved by 15.6% (average score before intervention: 67.3, post-intervention: 77.8), while the control group improved by 9.2% (average score before intervention: 66.8, post-intervention: 73.0). After testing, there was a certain difference ($F(1,52) = 3.98, p = 0.05, \eta^2 = 0.07$). This type of student values the feedback provided by digital twins to improve their skills, as

student E9 said, “Digital twins indicate that my function is inefficient, so I learned to simplify it.”

For the group of high-talent students, both subgroups showed a performance improvement; the experimental subgroup showed a 10.8% increase in scores (with an average score of 78.5 before intervention and 87.0 after intervention). In comparison, the control subgroup showed a 7.5% increase (with an average score of 79.1 before intervention and 85.0 after intervention). After testing ($F(1,38) = 2.11, p = 0.15, \eta^2 = 0.05$), the difference between the two groups was not significant. However, these talented students often use the collaborative capabilities of digital twins to explore more advanced concepts, as student E4 said, “We have designed a more complex robot path by integrating various functions—something that cannot be attempted in traditional laboratories.”

4.4. Qualitative Findings

The thematic analysis of interviews and study logs identified three key themes.

Theme 1: DT visualization bridges the gap between abstraction and concreteness.

Students in the experimental group often noticed that seeing code come to life in the virtual smart classroom helps them grasp abstract concepts more effectively. For example, a student struggling with functions explained: “I did not get why we need parameters until I programmed the robot to move different distances—changing the number in the function made it go farther” (Student E17). Control group students more often described concepts as “just words on a page” (Student C5).

Theme 2: Real-time feedback enhances learning iteration.

DT users emphasized the value of immediate, contextual feedback: “When the DT said ‘sensor data not read’, I knew to check my input function instead of guessing” (Student E8). Control group students, relying on delayed feedback, reported frustration: “I would forget what I was thinking when I wrote the code by the time I got it back” (Student C14).

Theme 3: Low-stakes experimentation reduces anxiety.

Emotional engagement was closely tied to the DT’s sense of a safe environment. As one student put it: “In traditional labs, I was scared to try new code because I might break something. In the DT, I can mess up and just reset—so I take more risks” (Student E19). This kind of adventurous behavior often leads to deeper learning as students explore edge cases that they would not otherwise test (such as “What if the loop runs zero times?”).

5. Discussion

5.1. Enhancing Engagement Through Digital Twins

The findings confirm that digital twin technology has a significant impact on learning engagement in introductory programming, with substantial effects on behavioral, cognitive, and emotional dimensions. These results align with self-determination theory [45], which posits that engagement is driven by three psychological needs, autonomy, competence, and relatedness, all of which are fulfilled by the DT environment. This section discusses the mechanism of increasing participation through digital twins. Figure 7 presents the mechanism of growing participation through digital twins, providing a visual theoretical framework to transform abstract theories into visual models that effectively explain the inherent logic of digital twins in increasing participation.

This concept diagram illustrates how the digital twin environment can enhance student engagement by meeting the three core needs of self-determination theory.

Autonomy: This is achieved through “independent task selection” and “custom virtual scenes”, cultivating greater behavioral engagement as students master their learning. The interactivity of DT enables students to guide their learning, from selecting robot actions to attempting alternative solutions. This sense of control does not exist in traditional “copy

and paste” laboratories, as it increases behavioral engagement: “I am not just doing what the teacher says—I have to decide what the robot does” (Student E7).

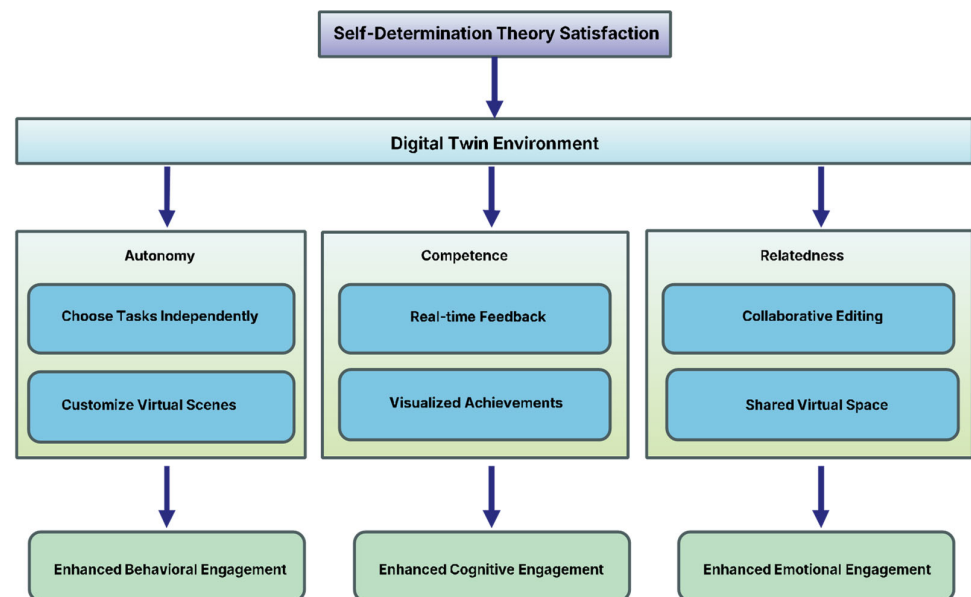


Figure 7. Conceptual diagram of mechanisms for digital twin-enhanced engagement.

Competence: This is supported by “Real-time Feedback” and “Visualized Achievements”, strengthening cognitive engagement through clear progress validation. The instant feedback in DT technology provides clear evidence of progress (such as “the robot is moving correctly—functioning properly!”) and cultivates a sense of mastery. This increases cognitive engagement, as students become motivated to tackle more complex tasks once they feel capable of doing so.

Relatedness: This is facilitated by “Collaborative Editing” and “Shared Virtual Space”, boosting emotional engagement via social connection. The DT’s collaborative space allowed students to share challenges and successes, fulfilling the need for social connection. This reduced feelings of isolation, often reported in programming courses, enhancing emotional engagement [46].

Notably, emotional engagement showed the most significant relative improvement, suggesting that DTs address a critical pain point in programming education: anxiety. The safe, consequence-free environment of the DT reduced the fear of failure, encouraging persistence. As a student once said, “I used to give up when the code did not work, now I think, ‘Let me try to fix it’, because DT told me where to look” (Student E11). This finding is consistent with relevant research, which has shown that positive emotional experiences generated during the learning process are often associated with higher memory retention rates.

5.2. Skill Development Mechanisms

The excellent performance of the experimental group in skill mastery fully demonstrates the significant effectiveness of digital twin (DT) technology in cultivating students’ practical programming abilities. Three mechanisms can explain this result.

(1) Instant and situational feedback

The real-time feedback loop of DT technology, where code changes immediately alter the state of the virtual system, enables rapid knowledge correction. This is consistent with the iterative learning model [47], which emphasizes that skill mastery requires frequent and timely feedback. For example, students who declare variables incorrectly in a DT will

see the virtual sensor display “undefined”, which serves as a direct prompt to reexamine the variable syntax. This is in stark contrast to traditional environments, where feedback is often delayed and disconnected from its context, thereby disrupting the connection between action and outcome.

(2) Visualization of abstract logic

A DT’s animation of algorithm flows (such as loops and data processing) helps students develop computational thinking and the ability to break down problems into logical steps. This is especially beneficial for students with a low ability, as it provides an alternative to pure symbolic reasoning. This supports the cognitive load theory [35], which suggests that allocating information through visual and textual channels can reduce mental labor and free up resources for deeper learning.

(3) Authentic problem-solving

A DT’s simulation of real-world scenarios (such as programming for the virtual smart classroom) makes coding tasks relevant and motivates students to develop transferable skills. For example, when debugging the motion algorithm of a robot assistant, not only is grammar knowledge required but logical reasoning ability also is—this is the core ability in the field of professional programming. This is in stark contrast to traditional textbook exercises, which often appear rigid and deliberate, making it challenging to cultivate skills that can be transferred to practical scenarios [48].

5.3. Addressing Equity in Programming Education

The significant progress of low-talent students suggests that digital twins may help narrow the achievement gap in programming education. Traditional teaching often fails to benefit students with limited abstract reasoning abilities, as they struggle to transform symbolic codes into mental models [49]. The multimodal representation of DT technology (visual and interactive) offers an alternative approach to understanding, aligning with the design of a universal learning framework [50], which advocates for multiple representation methods to accommodate diverse learners.

For example, a student with low spatial reasoning ability may struggle to visualize cyclic behavior in their mind. However, they can master it by observing the iterations of virtual robots. This ease of operation is of great significance, as programming education has long excluded students with different learning needs [51]. Digital twins (DTs) create a more inclusive learning environment by offering various participatory concepts.

It is worth mentioning that high-talent students can also utilize digital twins to explore advanced applications, but the improvement is relatively small. This suggests that digital twins can cater to the diverse needs of learners, laying a foundation for beginners while providing advanced project resources for more experienced students.

5.4. Limitations

Several limitations are worth noting when interpreting the research results in this study. Firstly, the expected duration of the intervention is relatively short, only 8 weeks, which suggests that students’ long-term retention of skills may be insufficient. Although the digital twin group has shown immediate results, further research still needs to track students’ performance in subsequent courses (such as “data structures”) to evaluate the transferability of skills. Secondly, there are specific obstacles related to technology and cost. This DT platform not only requires moderate computing power support, such as 8GB of memory and dedicated graphics cards, but also requires two developers to spend about 200 h on development, which may limit its promotion and application in resource-limited institutions. In addition, the research sample only comes from one university, whose

demographic data reflect trends in the region, such as gender distribution. Therefore, the research findings may be difficult to generalize to other scenarios, such as online courses or institutions with more diverse student populations. Additionally, a potential novelty effect suggests that the initial increase in participation by the digital twin population may be partially due to the novelty of the technology. Therefore, it is necessary to conduct longitudinal research to investigate whether students' learning motivation remains high after long-term use.

5.5. Future Research Directions

To overcome the limitations of existing research and deepen research results, future exploration can be carried out from multiple aspects: Firstly, we should focus on the impact of digital twin-based learning models on long-term outcomes, specifically tracking their roles in advanced programming course performance and career preparation, to clarify the sustained value of this learning model. Secondly, efforts should be made to develop digital twin platforms that rely on networks or mobile devices. These platforms should minimize hardware requirements so that educational institutions with relatively scarce resources can easily use them and expand the application scope of technology. In addition, artificial intelligence can be combined with digital twin technology to achieve personalized feedback, such as dynamically adjusting the learning difficulty based on students' error patterns, providing targeted concept reviews for students' weak links, and improving the learning efficiency of different learners. Finally, to comprehensively consider the universal applicability of the research conclusions, existing research results still need to be validated in online courses, vocational training programs, and various institutions with a more diverse student composition.

6. Conclusions and Recommendations

6.1. Conclusions

Digital twin technology has had a significant positive impact on learning participation in the "Programming Fundamentals" course, as reflected in multiple dimensions, including behavior, cognition, and emotion. Emotional engagement showed the most significant improvement, as the DT's secure environment reduced coding anxiety. The learning environment built on digital twins (DTs) can effectively enhance students' mastery of core programming skills, especially in algorithm design and code debugging. This is due to the real-time visualization of the code running process, the feedback mechanism that fits the scene, and the integration of real-world problem-solving tasks. It is worth noting that low-talent students benefit more significantly from digital twin technology, which means that digital technology is expected to narrow the gap in academic performance and provide strong support for the fairness of programming education.

6.2. Teaching Recommendations

In terms of course integration, 30% to 50% of traditional laboratory time can be replaced with activities based on digital twins, with a focus on conceptual visualization, such as using virtual sensors to explain variables and making abstract variable concepts more intuitive through changes in sensor data. Simultaneously, conducting interactive debugging, such as tracking loop errors through robot movements, links logical issues in the code with the actual actions of the robot, making the debugging process easier to understand. It is also necessary to integrate project-based learning, guide students to write virtual system code related to the real world, and enhance the practicality of learning. In differentiated teaching, the adaptability of digital twins should be utilized to tailor tasks according to the needs of different learners. For students with low talent, a simplified virtual

system should be provided, such as one that involves only one robot action, supplemented by guided feedback to help them gradually master the knowledge. For high-talent students, complex scenarios such as open challenges with multi-sensor integration are designed to stimulate their curiosity and innovation ability. For instructor training, it is necessary to equip educators with the ability to design digital twin activities that align with course objectives, such as linking robot motion with the concept of circulation, to make teaching activities more targeted. We also need to learn how to interpret the data generated by digital twins, such as analyzing common error patterns, to identify students' misunderstandings and adjust teaching accordingly and promptly. At the same time, this should also promote collaborative learning in the digital twin environment such as guiding students to conduct peer code review activities to cultivate their cooperation and communication skills.

6.3. Final Remarks

Digital twin technology represents a transformative approach to “Fundamentals of Programming” education, addressing key challenges in participation and skill acquisition. By creating an immersive interactive learning environment that reflects the real-world programming environment, DT bridges the gap between abstract code and tangible results. This not only enhances current learning outcomes but also cultivates the perseverance and algorithmic thinking necessary for long-term development in the field of computer science. With the continuous advancement of educational technology, digital twins have outlined an inclusive, interactive, and practical blueprint for programming education, enabling students not only to write code proficiently but also to thrive and excel in this technology-driven world.

Author Contributions: Conceptualization, M.L. and Z.H.; methodology, M.L.; software, M.L.; validation, M.L. and Z.H.; formal analysis, M.L.; investigation, M.L.; resources, Z.H.; data curation, M.L.; writing—original draft preparation, M.L.; writing—review and editing, M.L.; visualization, M.L.; supervision, Z.H.; project administration, Z.H.; funding acquisition, M.L. All authors have read and agreed to the published version of the manuscript.

Funding: M.L. has been awarded funding from the Collaborative Education Project of the Ministry of Education of China (Project No. 220903880215602) and the Teaching Research and Reform Project of Wenzhou University (Project No. jg2024046). This includes the article processing fee (APC) and revision of the English manuscript.

Data Availability Statement: The data presented in this study are available on request from the corresponding author. (Due to privacy or ethical limitations, this data is not publicly available).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Nicolajsen, S.M.; Nielsen, S.; Carlsen, L.M.; Brabrand, C. Programming education across disciplines: A nationwide study of Danish higher education. *High. Educ.* **2024**, 1–27. [\[CrossRef\]](#)
2. Lee, H.Y.; Lin, C.J.; Wang, W.S.; Chang, W.C.; Huang, Y.M. Precision education via timely intervention in K-12 computer programming course to enhance programming skill and affective-domain learning objectives. *Int. J. STEM Educ.* **2023**, *10*, 52. [\[CrossRef\]](#)
3. Groher, I.; Vierhauser, M.; Sabitzer, B.; Kuka, L.; Hofer, A.; Muster, D. Exploring diversity in introductory programming classes: An experience report. In Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Software Engineering Education and Training, Pittsburgh, PA, USA, 22–24 May 2022. [\[CrossRef\]](#)
4. Giannakoulas, A.; Xinogalos, S. Studying the effects of educational games on cultivating computational thinking skills to primary school students: A systematic literature review. *J. Comput. Educ.* **2024**, *11*, 1283–1325. [\[CrossRef\]](#)
5. Zhan, Q.; Wang, J.; Pan, X.; Ding, Y.; Liu, Y. Teaching Model Design of Computer Programming Courses for Digital Media Technology Students. *Wirel. Commun. Mob. Comput.* **2022**, *2022*, 7085914. [\[CrossRef\]](#)

6. LoSchiavo, F.M. How to Create Automatically Graded Spreadsheets for Statistics Courses. *Teach. Psychol.* **2016**, *43*, 147–152. [\[CrossRef\]](#)
7. She, M.; Xiao, M.; Zhao, Y. Technological Implication of the Digital Twin Approach on the Intelligent Education System. *Int. J. Hum. Robot.* **2023**, *20*, 2250005. [\[CrossRef\]](#)
8. Tao, F.; Cheng, J.; Qi, Q.; Zhang, M.; Zhang, H.; Sui, F. Digital twin-driven product design, manufacturing and service with big data. *Int. J. Adv. Manuf. Technol.* **2018**, *94*, 3563–3576. [\[CrossRef\]](#)
9. Deren, L.; Wenbo, Y.; Zhenfeng, S. Smart city based on digital twins. *Comput. Urban. Sci.* **2021**, *1*, 4. [\[CrossRef\]](#)
10. Tarng, W.; Wu, Y.J.; Ye, L.Y.; Tang, C.W.; Lu, Y.C.; Wang, T.L.; Li, C.L. Application of Virtual Reality in Developing the Digital Twin for an Integrated Robot Learning System. *Electronics* **2024**, *13*, 2848. [\[CrossRef\]](#)
11. Dihan, M.S.; Akash, A.I.; Tasneem, Z.; Das, P.; Das, S.K.; Islam, M.R.; Islam, M.M.; Badal, F.R.; Ali, M.F.; Ahamed, M.H.; et al. Digital twin: Data exploration, architecture, implementation and future. *Heliyon* **2024**, *10*, e26503. [\[CrossRef\]](#)
12. Fan, G.; Liu, D.; Zhang, R.; Pan, L. The impact of AI-assisted pair programming on student motivation, programming anxiety, collaborative learning, and programming performance: A comparative study with traditional pair programming and individual approaches. *Int. J. STEM Educ.* **2025**, *12*, 16. [\[CrossRef\]](#)
13. Maloney, J.; Resnick, M.; Rusk, N.; Silverman, B.; Eastmond, E. The scratch programming language and environment. *ACM Trans. Comput. Educ.* **2010**, *10*, 1–15. [\[CrossRef\]](#)
14. Kroustalli, C.; Xinogalos, S. Studying the effects of teaching programming to lower secondary school students with a serious game: A case study with Python and CodeCombat. *Educ. Inf. Technol.* **2021**, *26*, 6069–6095. [\[CrossRef\]](#)
15. Dong, Z.; Han, X.; Shi, Y.; Zhai, W.; Luo, S. Development of Manipulator Digital Twin Experimental Platform Based on RCP. *Electronics* **2022**, *11*, 4196. [\[CrossRef\]](#)
16. Hananto, A.L.; Tirta, A.; Herawan, S.G.; Idris, M.; Soudagar, M.E.M.; Djamari, D.W.; Veza, I. Digital Twin and 3D Digital Twin: Concepts, Applications, and Challenges in Industry 4.0 for Digital Twin. *Computers* **2024**, *13*, 100. [\[CrossRef\]](#)
17. Iliuță, M.-E.; Moisesescu, M.-A.; Pop, E.; Ionita, A.-D.; Caramihai, S.-I.; Mitulescu, T.-C. Digital Twin—A Review of the Evolution from Concept to Technology and Its Analytical Perspectives on Applications in Various Fields. *Appl. Sci.* **2024**, *14*, 5454. [\[CrossRef\]](#)
18. Ye, W.; Liu, X.; Zhao, X.; Fu, H.; Cai, Y.; Li, H. A Data-Driven Digital Twin Architecture for Failure Prediction of Customized Automatic Transverse Robot. *IEEE Access* **2024**, *12*, 59222–59235. [\[CrossRef\]](#)
19. Holman, T. The best of CodePen. *Net* **2016**, *283*, 76–81.
20. Wielemaker, J.; Lager, T.; Riguzzi, F. SWISH: SWI-Prolog for Sharing. *arXiv* **2015**. [\[CrossRef\]](#)
21. Pitzalis, R.F.; Giordano, A.; Di Spigno, A.; Cowell, A.; Niculita, O.; Berselli, G. Application of Augmented Reality-Based Digital Twin Approaches: A Case Study in Industrial Equipment. *Int. J. Adv. Manuf. Technol.* **2025**, *138*, 3747–3763. [\[CrossRef\]](#)
22. Filipescu, A.; Simion, G.; Ionescu, D. IoT-Cloud, VPN, and Digital Twin-Based Remote Monitoring and Control of a Multifunctional Robotic Cell in the Context of AI, Industry, and Education 4.0 and 5.0. *Sensors* **2024**, *24*, 7451. [\[CrossRef\]](#)
23. Erceylan, G.; Akbarzadeh, A.; Gkioulos, V. Leveraging digital twins for advanced threat modeling in cyber-physical systems cybersecurity. *Int. J. Inf. Secur.* **2025**, *24*, 151. [\[CrossRef\]](#)
24. Al Hakim, V.G.; Yang, S.H.; Wang, J.H.; Lin, H.H.; Chen, G.D. Digital Twins of Pet Robots to Prolong Interdependent Relationships and Effects on Student Learning Performance. *IEEE Trans. Learn. Technol.* **2024**, *17*, 1883–1897. [\[CrossRef\]](#)
25. Fredricks, J.A.; Blumenfeld, P.C.; Paris, A.H. School engagement: Potential of the concept, state of the evidence. *Rev. Educ. Res.* **2004**, *74*, 59–109. [\[CrossRef\]](#)
26. Toukiloglou, P.; Xinogalos, S. Effects of Collaborative Support on Learning in Serious Games for Programming. *J. Educ. Comput. Res.* **2025**, *63*, 126–146. [\[CrossRef\]](#)
27. Gebre, E.; Saroyan, A.; Bracewell, R. Students' engagement in technology rich classrooms and its relationship to professors' conceptions of effective teaching. *Br. J. Educ. Technol.* **2014**, *45*, 83–96. [\[CrossRef\]](#)
28. Hong, W.; Zhen, R.; Liu, R.D.; Wang, M.T.; Ding, Y.; Wang, J. The longitudinal linkages among Chinese children's behavioural, cognitive, and emotional engagement within a mathematics context. *Educ. Psychol.* **2020**, *40*, 666–680. [\[CrossRef\]](#)
29. Silva, L.; Mendes, A.; Gomes, A.; Fortes, G. What Learning Strategies are Used by Programming Students? A Qualitative Study Grounded on the Self-regulation of Learning Theory. *ACM Trans. Comput. Educ.* **2024**, *24*, 9. [\[CrossRef\]](#)
30. Berry, C.; Walcott, M. Reducing Caribbean's Students' "Code-Phobia" with Programming in Scratch. In Proceedings of the 2019 ACM Conference on International Computing Education Research, Toronto, ON, Canada, 12–14 August 2019. [\[CrossRef\]](#)
31. Vishnu, S.; Sahil; Garg, N. Unveiling the Role of GPT-4 in Solving LeetCode Programming Problems. *Comput. Appl. Eng. Educ.* **2025**, *33*, e22815. [\[CrossRef\]](#)
32. Bao, L.; Xing, Z.; Xia, X.; Lo, D. VT-Revolution: Interactive Programming Video Tutorial Authoring and Watching System. *IEEE Trans. Softw. Eng.* **2019**, *45*, 823–838. [\[CrossRef\]](#)
33. Wang, Y.; Liang, Y.; Liu, L.; Liu, Y. A multi-peer assessment platform for programming language learning: Considering group non-consensus and personal radicalness. *Interact. Learn. Environ.* **2016**, *24*, 2011–2031. [\[CrossRef\]](#)

34. Yun, H.; Jun, M.B.G. Immersive and interactive cyber-physical system (I2CPS) and virtual reality interface for human involved robotic manufacturing. *J. Manuf. Syst.* **2022**, *62*, 234–248. [\[CrossRef\]](#)
35. Yu, J.H.; Chang, X.Z.; Liu, W.; Huan, Z. An online integrated programming platform to acquire students' behavior data for immediate feedback teaching. *Comput. Appl. Eng. Educ.* **2023**, *31*, 520–536. [\[CrossRef\]](#)
36. Saville, B.K.; Pope, D.; Lovaas, P.; Williams, J. Interteaching and the Testing Effect: A Systematic Replication. *Teach. Psychol.* **2012**, *39*, 280–283. [\[CrossRef\]](#)
37. Roediger, H.L.; Karpicke, J.D. The power of testing memory: Basic research and implications for educational practice. *Perspect. Psychol. Sci.* **2006**, *1*, 181–210. [\[CrossRef\]](#)
38. Kirschner, P.A.; Sweller, J.; Kirschner, F.; Zambrano, R.J. From Cognitive Load Theory to Collaborative Cognitive Load Theory. *Int. J. Comput.-Support. Collab. Learn.* **2018**, *13*, 213–233. [\[CrossRef\]](#)
39. Zourmpakis, A.I.; Kalogiannakis, M.; Papadakis, S. The Effects of Adaptive Gamification in Science Learning: A Comparison Between Traditional Inquiry-Based Learning and Gender Differences. *Computers* **2024**, *13*, 324. [\[CrossRef\]](#)
40. Appleton, J.J.; Christenson, S.L.; Furlong, M.J. Student engagement with school: Critical conceptual and methodological issues of the construct. *Psychol. Sch.* **2008**, *45*, 369–386. [\[CrossRef\]](#)
41. Matosas-López, L.; Leguey-Galán, S.; Regaña, C.B.; Piris, N.P. University and Quality Systems. Evaluating faculty performance in face-to-face and online programs: A comparison of Likert and BARS instruments. *Int. J. Educ. Res. Innov.* **2024**, *22*, 18. [\[CrossRef\]](#)
42. Braun, V.; Clarke, V. Is thematic analysis used well in health psychology? A critical review of published research, with recommendations for quality practice and reporting. *Health Psychol. Rev.* **2023**, *17*, 695–718. [\[CrossRef\]](#)
43. Moon, S.; Yeo, G.; Kim, Y.; Oh, J. A study on the consumer behavior and attitude toward low-sodium convenience store foods. *Nutr. Res. Pract.* **2024**, *18*, 567–585. [\[CrossRef\]](#)
44. Faul, F.; Erdfelder, E.; Lang, A.G.; Buchner, A. G*Power 3: A flexible statistical power analysis program for the social, behavioral, and biomedical sciences. *Behav. Res. Methods* **2007**, *39*, 175–191. [\[CrossRef\]](#) [\[PubMed\]](#)
45. Ryan, R.M.; Deci, E.L. Self-determination theory and the facilitation of intrinsic motivation, social development, and well-being. *Am. Psychol.* **2000**, *55*, 68–78. [\[CrossRef\]](#) [\[PubMed\]](#)
46. Goetz, T.; Frenzel, A.C.; Pekrun, R.; Hall, N.C.; Lüdtke, O. Between- and within-domain relations of students' academic emotions. *J. Educ. Psychol.* **2007**, *99*, 715–733. [\[CrossRef\]](#)
47. Robins, A.; Rountree, J.; Rountree, N. Learning and Teaching Programming: A Review and Discussion. *Comput. Sci. Educ.* **2003**, *13*, 137–172. [\[CrossRef\]](#)
48. Mccracken, M.; Wilusz, T.; Almstrum, V.; Diaz, D.; Guzdial, M.; Hagan, D.; Kolikant, Y.B.; Laxer, C.; Thomas, L.; Utting, I. A multi-national, multi-institutional study of assessment of programming skills of first-year CS students. In Proceedings of the 6th Annual Conference on Innovation and Technology in Computer Science Education, Canterbury, UK, 1 December 2001. [\[CrossRef\]](#)
49. Bosse, Y.; Gerosa, M.A. Difficulties of Programming Learning from the Point of View of Students and Instructors. *IEEE Lat. Am. Trans.* **2017**, *15*, 2191–2199. [\[CrossRef\]](#)
50. Morgan, A. Enhancing Access in an Online Course Using Universal Design for Learning (UDL) and Scenario-Based Learning (SBL). *TechTrends* **2024**, *68*, 904–913. [\[CrossRef\]](#)
51. Fisher, A.; Margolis, J. Unlocking the clubhouse: Women in computing. In Proceedings of the 34th SIGCSE Technical Symposium on Computer Science Education, Nevada, NV, USA, 19–23 February 2003. [\[CrossRef\]](#)

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.